Office Visitor and Task Management System (OVTMS)

Object-Oriented Programming (OOP) Implementation Version (Without Laravel)

Revised Implementation Approach

This version transforms the project from a Laravel framework implementation into a **pure Object-Oriented PHP system** using:

- PHP (Object-Oriented Programming)
- HTML5
- CSS3
- Bootstrap
- JavaScript
- MySQL
- Git
- Apache (XAMPP/LAMP)

Architecture Pattern:

Use **Object-Oriented Modular Architecture (MVC-like structure without framework)**.

1. Proposed Project Structure

OVTMS/
│

```
├── config/
│   └── Database.php
│
├── classes/
│   ├── User.php
│   ├── Role.php
│   ├── Visitor.php
│   ├── Task.php
│   ├── FollowUp.php
│   ├── Notification.php
│   ├── Report.php
│   └── Auth.php
│
├── controllers/
│   ├── LoginController.php
│   ├── UserController.php
│   ├── VisitorController.php
│   ├── TaskController.php
│   ├── FollowUpController.php
│   ├── ReportController.php
│   └── DashboardController.php
│
├── models/
│   ├── UserModel.php
│   ├── VisitorModel.php
│   ├── TaskModel.php
│   └── ...
│
├── views/
│   ├── login.php
│   ├── admin/
│   ├── supervisor/
│   ├── staff/
│   └── receptionist/
│
```

```
├── includes/
│   ├── header.php
│   ├── sidebar.php
│   └── footer.php
│
├── assets/
│   ├── css/
│   ├── js/
│   └── bootstrap/
│
└── index.php
```

---

2. Core System Classes

Authentication Class

Class:

Auth.php

Responsibilities:

Methods:

login()
logout()
validateUser()
checkRole()
startSession()

Handles:

- Authentication
- Sessions
- Access control

User Class

User.php

Attributes:

- userId
- name
- email
- password
- roleId
- status

Methods:

createUser()
updateUser()
deleteUser()
getUsers()

Role Class

Role.php

Methods:

assignRole()
checkPermissions()

Roles:

- Admin
- Supervisor
- Staff
- Receptionist

Visitor Class

Visitor.php

Methods:

registerVisitor()
checkIn()
checkOut()
assignHost()
recordOutcome()

Implements Visitor Management Module.

Task Class

Task.php

Methods:

createTask()
assignTask()
updateTask()

checkOverdue()
completeTask()

Implements Task Management Module.

FollowUp Class

FollowUp.php

Methods:

createFollowUp()
assignAction()
reviewFollowUp()
closeCase()

Notification Class

Notification.php

Methods:

sendNotification()
markRead()
getAlerts()

Report Class

Report.php

Methods:

generateVisitorReport()
generateTaskReport()
generatePerformanceReport()
exportPDF()

3. Database Connection Class

Database.php

```
class Database {
private $host='localhost';
private $dbname='ovtms';
private $user='root';
private $password='';
public $conn;

public function connect(){
$this->conn = new mysqli(
$this->host,
$this->user,
$this->password,
$this->dbname
);
return $this->conn;
}
}
```

Reusable by all classes.

4. Modules

Module 1 Authentication Module

Classes:

- Auth
- User
- Role

Processes:

- Login
- Role validation
- Session management
- Logout

Pages:

- login.php
- authenticate.php

Input
- Email
- password

Module 2 Administration Module

Classes:

- User
- Role

- Report

Functions:

- Manage users
- System settings
- View reports

Admin Dashboard:

admin/dashboard.php

Module 3 Visitor Management Module

Classes:

- Visitor
- Notification
- FollowUp

Functions:

- Register visitor
- Check-in
- Assign host
- Notify staff
- Check-out
- Trigger follow-up

Pages:

visitors/register.php
visitors/checkin.php

visitors/checkout.php

Input

-

Module 4 Task Management Module

Classes:

- Task
- Notification

Functions:

- Create tasks
- Set priorities
- Assign tasks
- Update progress
- Overdue checks
- Complete tasks

Pages:

tasks/create.php
tasks/list.php
tasks/update.php

Module 5 Follow-Up Module

Class:

- FollowUp

Functions:

- Create follow-up
- Assign action
- Review
- Escalate
- Close case

Module 6 Notifications Module

Class:

- Notification

Functions:

- Email alerts
- Dashboard alerts
- Overdue reminders

Module 7 Reports Module

Class:

- Report

Reports:

- Daily Visitor Report
- Task Status Report
- Staff Performance Report

Module 8 Dashboard Module

Use role-based dashboards.

Files:

admin/dashboard.php
supervisor/dashboard.php
staff/dashboard.php
reception/dashboard.php

Widgets:

- Total visitors
- Pending tasks
- Overdue alerts
- Completed tasks

5. OOP Relationships (Class Diagram Logic)

```
User
 ├── Admin
 ├── Supervisor
 ├── Staff
 └── Receptionist

User ---- creates ---- Tasks
User ---- hosts ------ Visitors
Task ---- has -------- FollowUps
User ---- receives --- Notifications
Report -- reads ------ Tasks/Visitors
```

This is your object model.

6. Database Tables (Same ERD)

Tables remain:

- roles
- users
- visitors
- tasks
- followups
- notifications
- reports

Use OOP classes to perform CRUD on these.

7. Example OOP CRUD Pattern

Example Task Class:

```
class Task {
private $db;

public function __construct(){
$this->db=(new Database())->connect();
}

public function createTask($title,$priority){
$sql="INSERT INTO tasks(title,priority)
```

```
VALUES(?,?)";
}

public function getTasks(){ }

public function updateTask(){ }
}
```

8. Security Implementation in OOP

Implement:

- Password hashing

```
password_hash()
password_verify()
```

- Session authentication
- Prepared statements
- Input validation
- CSRF token handling
- Role authorization checks

9. Suggested OOP Development Order

Phase 1 Foundation

Build:

1 Database class
2 Authentication class
3 User and Role classes
4 Login system
5 Role dashboards

Phase 2 Core Modules

Build:

6 Visitor class
7 Task class
8 FollowUp class

Phase 3 Support Modules

Build:

9 Notification class
10 Report class
11 Dashboard metrics

Phase 4 Security and Testing

Build:

12 Audit logs
13 Validation
14 Testing

10. Mapping SRS Modules to OOP Classes

SRS Module	OOP Class
Authentication	Auth
User Management	User
Roles	Role
Visitor Management	Visitor
Task Management	Task
Follow Up	FollowUp
Notifications	Notification
Reports	Report
Dashboards	DashboardController

---

11. Technologies Used

Frontend:

- HTML
- CSS

- JavaScript

Backend:

- Core PHP OOP

Database:

- MySQL/MariaDB

Version Control:

- Git

Server:

- Apache

Design Tool:

- Draw.io

12. Final Revised Implementation Architecture

Architecture:

Presentation Layer

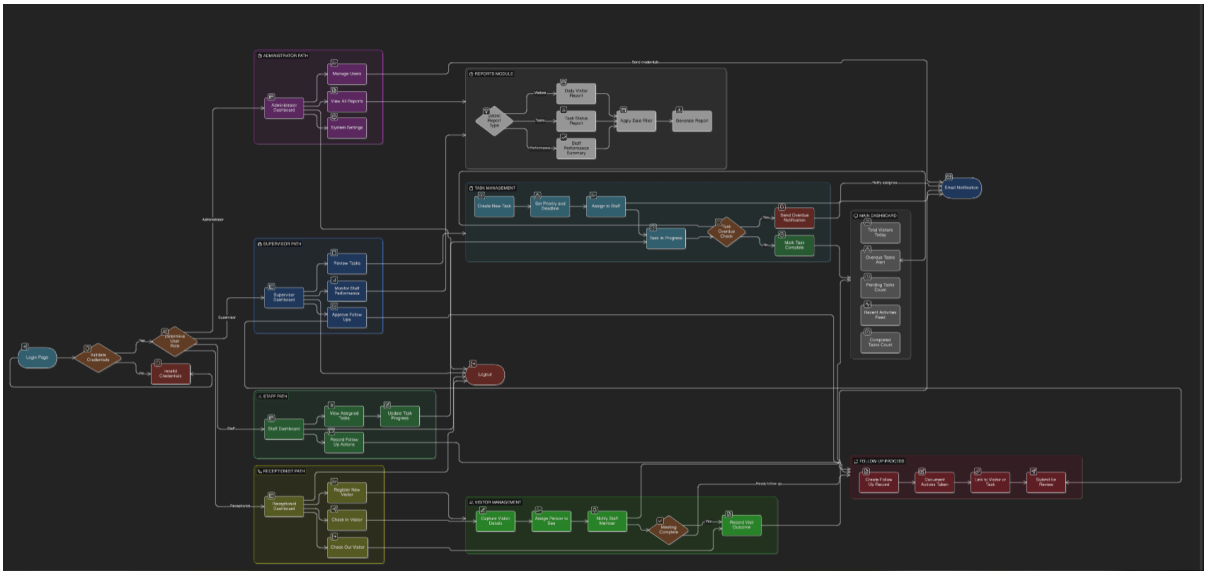
- HTML
- CSS

Business Logic Layer

- OOP PHP Classes
- Controllers

Data Access Layer

- Models
- Database Class
- MySQL

Three-layer architecture.

13.system architectural diagram